

CampusPulse 2026

A Cloud-Native Smart Campus Operations Platform Implementation Report

Big Data and Cloud Computing

EPITA, academic year 2025-2026

Amazon Web Services, Free Tier, region us-east-1

3 October 2026

Team

Alero Awani

Tanushka Indresh Gupta

Om-balu Kalbhor

Project links

Live dashboard: <https://dz1a13ncwbkm3.cloudfront.net>

Source code: <https://github.com/alero-awani/CampusPulse>

Public API: <https://tat8tra5ok.execute-api.us-east-1.amazonaws.com/health>

Demo sign in (staff role)

Email: staff@northbridge.edu

Password: demo1234

There is no public sign up. Accounts are created by an administrator in Cognito.

Contents

This report follows the structure required in section 10 of the project brief, with two extra sections on GitHub Actions and Terraform.

1. Introduction
 2. System Architecture
 3. Technology Stack
 4. Implementation Process
 5. Infrastructure as Code with Terraform
 6. CI/CD with GitHub Actions
 7. Security Design
 8. Scalability Strategy
 9. Cost Analysis
 10. Observability
 11. Disaster Recovery Strategy
 12. Testing and Evaluation
 13. Lessons Learned
 14. Conclusion
 15. References and Reused Sources
- Appendix A: Data model
- Appendix B: API endpoints
- Appendix C: Deployment guide

1. Introduction

1.1 The problem

NorthBridge University runs a busy campus and has almost no live picture of what is happening inside it. Staff cannot tell which rooms are full, which buildings are burning through energy, or whether a lab is overheating until someone complains.

Student service requests arrive by email and get lost. Nobody knows which ones are urgent. The operations team wanted one place to see all of it, and they wanted it cheap.

1.2 Objectives

We set out to build a working platform that does five things:

- **Ingest data:** sensor readings arrive continuously from rooms across campus.
- **Process it:** raw readings become room status, severity levels and alerts.
- **Store it:** everything is stored in DynamoDB and survives a region failure.
- **Show it:** staff see live occupancy, energy use, alerts and requests.
- **Secure it:** three roles, real tokens, no public sign up.

The whole thing had to stay inside the AWS Free Tier. That constraint shaped nearly every design decision in this report.

1.3 Our use case

We built the hybrid option from the brief: a campus operations dashboard that combines occupancy monitoring, energy alerting and maintenance request management in one product.

A simulator plays the part of the campus. It models buildings, rooms, class timetables and daily rhythms, then injects problems like overcrowding, overheating, water leaks, forced doors and energy spikes, so we can prove the backend detects them.

CampusPulse

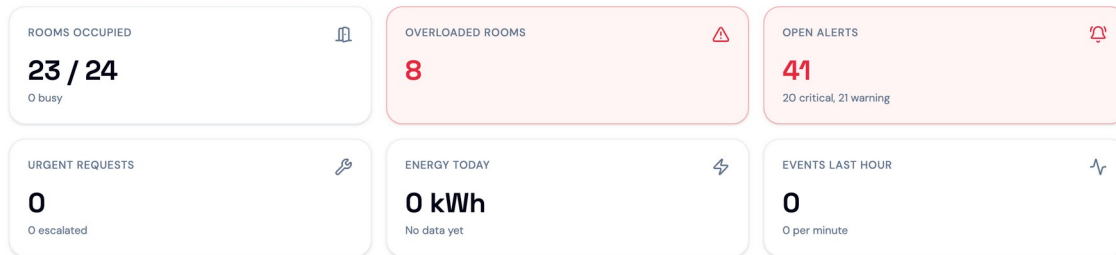
NorthBridge University

Overview Operations Energy

Updated 5s ago

Refresh

Jordan Lee Staff



Rooms

Building	Room	Type	Occupancy	Status	Temp	Humidity
Engineering-B	B101	Lecture hall	246/200	Overloaded	21.5 °C	43%
Library-A	A102	Study space	13/12	Overloaded	22.2 °C	46%
Library-A	A201	Library	102/80	Overloaded	22.1 °C	46%
Library-A	A301	Study space	27/20	Overloaded	22.7 °C	48%
Science-C	C203	Classroom	33/30	Overloaded	21.4 °C	44%
Science-C	C301	Study space	21/16	Overloaded	22.8 °C	45%
StudentHub-D	D103	Study space	12/10	Overloaded	22.5 °C	45%
StudentHub-D	D201	Classroom	53/45	Overloaded	21.6 °C	42%
Engineering-B	B102	Lab	3/30	Normal	36.2 °C	41%
Engineering-B	B103	Lab	2/24	Normal	21.5 °C	87%
Engineering-B	B201	Classroom	3/40	Normal	21.6 °C	44%
Engineering-B	B203	Classroom	8/35	Normal	36.3 °C	40%
Library-A	A101	Library	42/120	Normal	22.4 °C	79%
Library-A	A202	Study space	3/8	Normal	22.3 °C	45%
Library-A	A203	Study space	16/40	Normal	22.8 °C	69%
Science-C	C101	Lecture hall	10/150	Normal	21.4 °C	45%

Figure 1: The CampusPulse operations dashboard, showing live campus status.

2. System Architecture

2.1 The big picture

We built CampusPulse fully serverless, so there are no servers to patch, nothing idles overnight, and the bill drops to almost nothing when nobody uses it.

Requests take one of two paths through the system:

- Sensor path:** the simulator posts readings to POST /events with a device key, API Gateway passes them to the Lambda, the Lambda scores them against the threshold rules and writes events, alerts and counters to DynamoDB.
- User path:** the browser loads the dashboard from CloudFront, signs in through the Cognito hosted page, and calls the API with the resulting token. API Gateway checks the token before the Lambda ever runs.

CampusPulse 2026: smart campus operations platform

Northbridge University, AWS region us-east-1 (N. Virginia), Serverless, deployed with Terraform, designed for the AWS Free Tier. Disaster recovery: DynamoDB point-in-time recovery (35 days) and live replicates in eu-west-3 (Paris); S3 versioning and cross-region replication; everything rebuildable with Terraform.

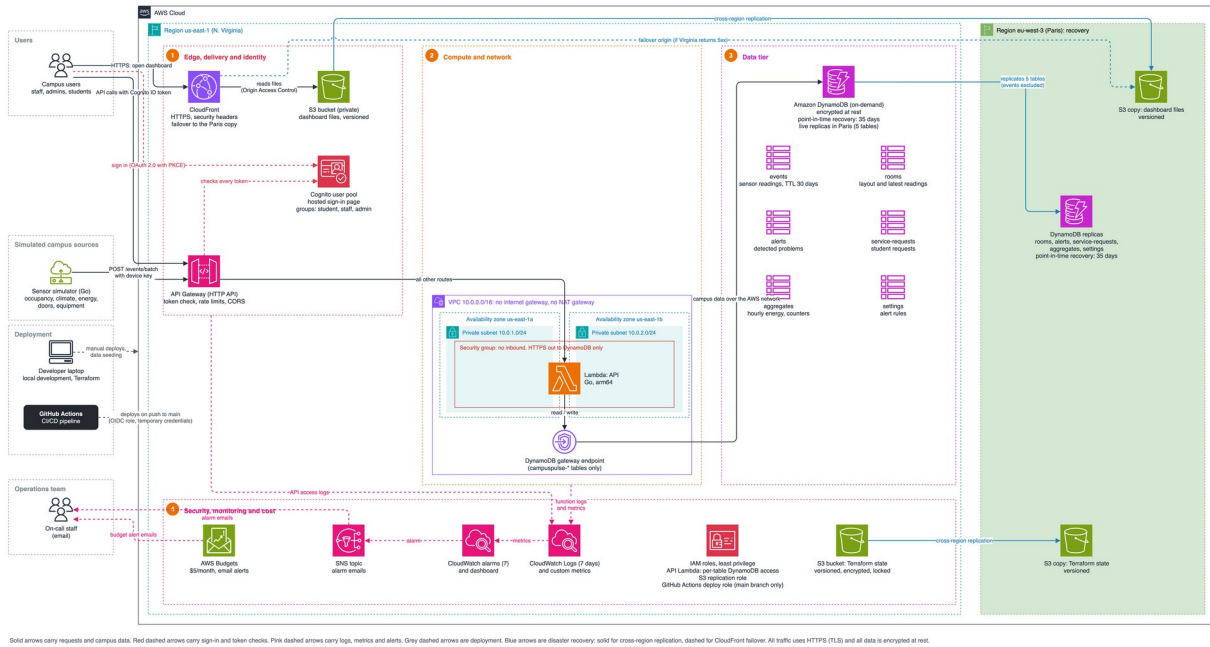


Figure 2: CampusPulse architecture: ingestion, compute, storage, security and monitoring.

2.2 What we plan to add next

The platform we built covers what the brief asked for, but it is a foundation rather than a finished product. The diagram below shows the capabilities we would add next, drawn against the same architecture so it is clear what changes and what stays.

CampusPulse 2026: what we plan to add next

Everything in grey, purple and pink is built and running today. The green band at the bottom is what we would add next, and none of it changes the shape of the architecture. NorthBridge University AWS region us-east-1 (N. Virginia). Serverless, deployed with Terraform, designed for the AWS Free Tier. Disaster recovery: DynamoDB point-in-time recovery (35 days) and five replicas in eu-west-3 (Paris); S3 versioning and cross-region replication; everything rebuildable with Terraform.

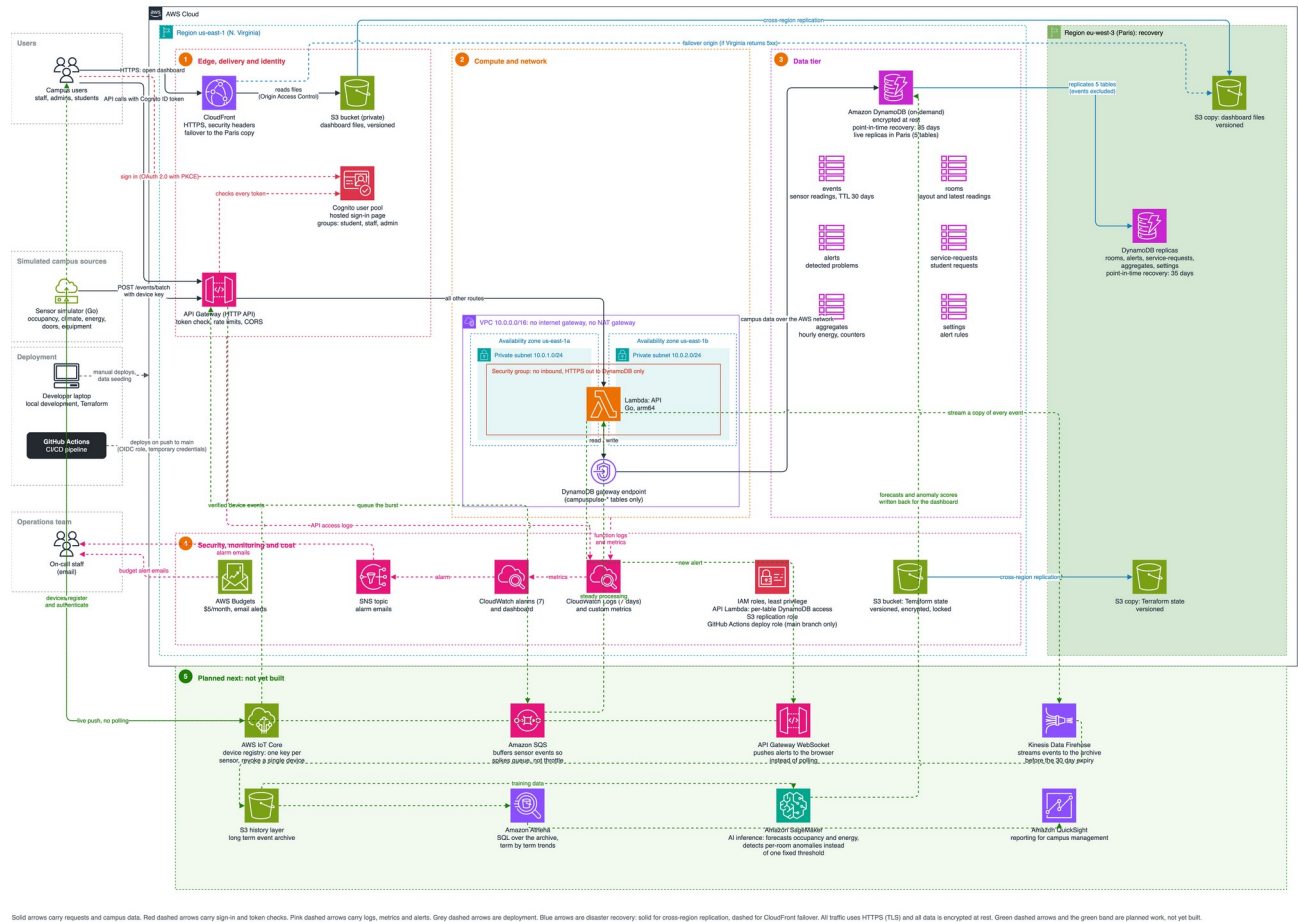


Figure 3: Planned future capabilities, shown against the current architecture.

In short, the capabilities we would add next are these:

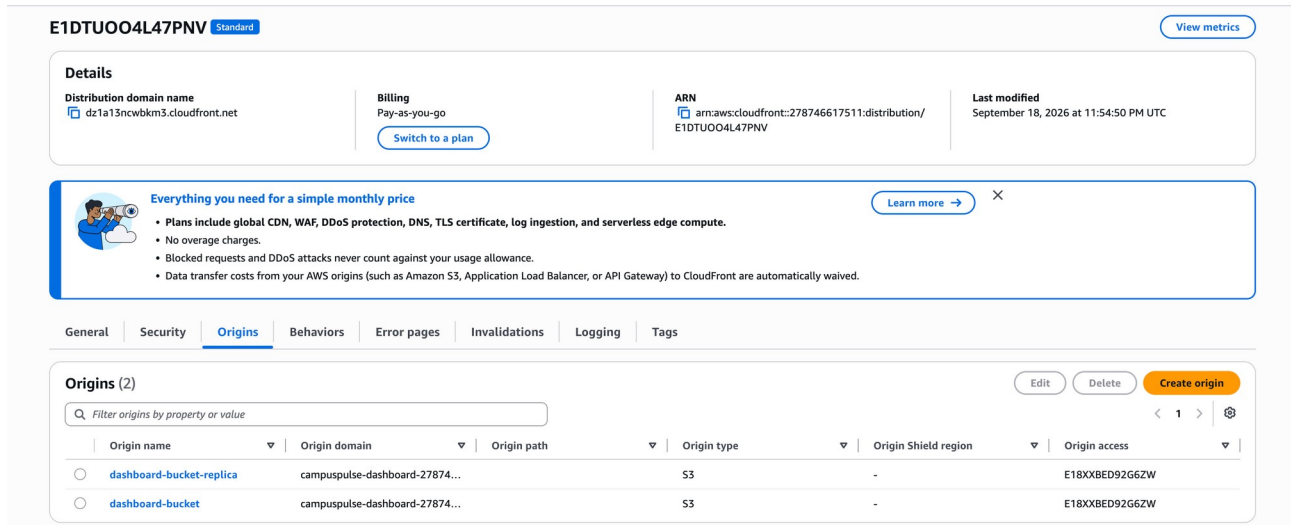
- **AI inference:** forecast occupancy and energy use from the history we already store, so the campus can plan ahead instead of reacting.
- **Anomaly detection:** learn what normal looks like per room, rather than comparing every reading against one fixed threshold.
- **A queue in front of ingestion:** buffer sensor events through SQS, so traffic spikes queue instead of being throttled.
- **Real device identity:** a registry with a key per device and the ability to revoke one, instead of a single shared key.
- **Live updates:** push alerts to the dashboard over WebSockets, instead of the browser polling every 30 seconds.
- **A history layer:** archive older events to S3 and query them with Athena, so trends survive the 30 day expiry without filling DynamoDB.

None of these change the shape of the architecture. They attach to what is already there, which was the point of keeping the services loosely coupled.

2.3 Layer by layer

Edge and delivery

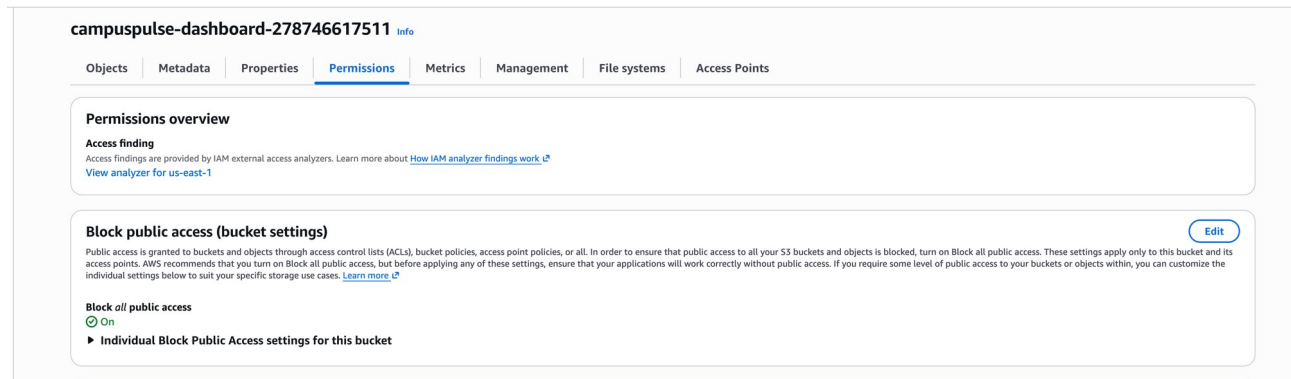
The dashboard is a static React build in an S3 bucket. The bucket blocks all public access. CloudFront reads it through an Origin Access Control, so the only way to reach the files is through the distribution, over HTTPS.



The screenshot shows the AWS CloudFront console for distribution **E1DTUOO4L47PNV** (Standard). The **Details** section includes the distribution domain name `dz1a13ncwbkm3.cloudfront.net`, billing method **Pay-as-you-go**, ARN `arn:aws:cloudfront::278746617511:distribution/E1DTUOO4L47PNV`, and last modified date **September 18, 2026 at 11:54:50 PM UTC**. A **Switch to a plan** button is also present. Below this is a promotional banner for CloudFront's features. The **Origins** tab is selected, showing two origins:

Origin name	Origin domain	Origin path	Origin type	Origin Shield region	Origin access
☐ dashboard-bucket-replica	campuspulse-dashboar-27874...		S3	-	E18XXBED92G6ZW
☐ dashboard-bucket	campuspulse-dashboar-27874...		S3	-	E18XXBED92G6ZW

Figure 4: CloudFront distribution E1DTUOO4L47PNV serving the dashboard over HTTPS.



The screenshot shows the AWS IAM console for the bucket **campuspulse-dashboar-278746617511**. The **Permissions** tab is selected. The **Permissions overview** section shows **Access finding** with a link to [View analyzer for us-east-1](#). The **Block public access (bucket settings)** section shows **Block all public access** set to **On**, with a link to [Individual Block Public Access settings for this bucket](#).

Figure 5: The dashboard bucket is completely private. Only CloudFront can read it.

API exposure

API Gateway is an HTTP API, not a REST API, because it costs about a third as much and we did not need the extra features. It handles CORS itself, throttles at 20 requests per second with a burst of 50, and writes a structured access log for every call.

Routes split into two groups. GET `/health` and the two sensor routes are public, with the Lambda checking the device key. Everything else goes through the `$default` route, which requires a valid Cognito token checked by the gateway.

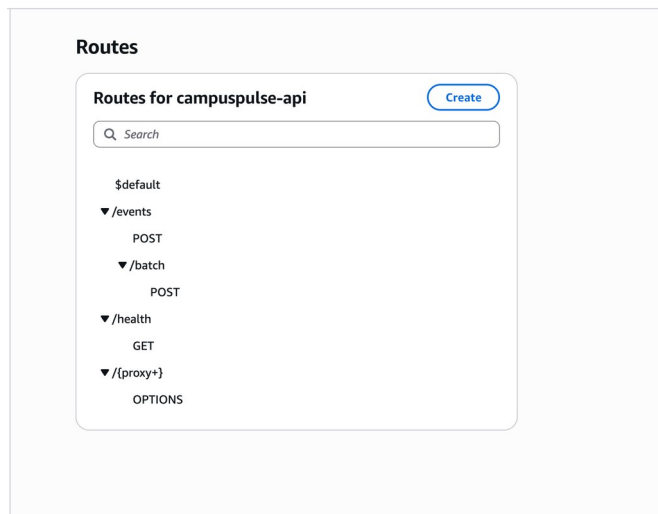


Figure 6: API Gateway routes. The \$default route carries the JWT authorizer.

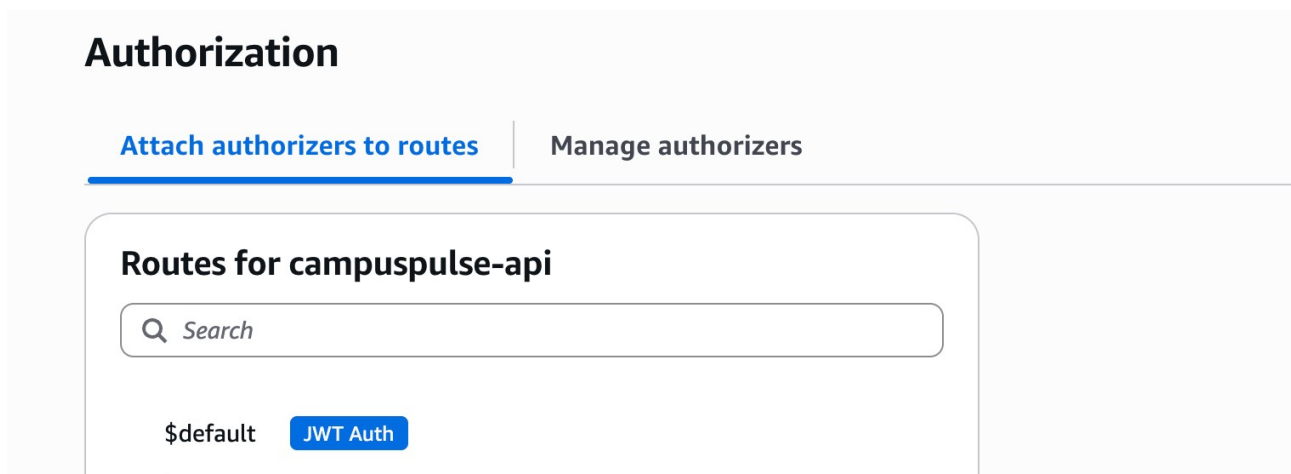


Figure 7: The Cognito JWT authorizer, accepting only tokens from our user pool and app client.

Identity

We used Amazon Cognito as the identity layer, so that we never store a password or write token code ourselves. It holds the accounts, runs the hosted sign in page, and issues the tokens the dashboard sends to the API.

The three roles are Cognito groups, and the group travels inside the signed token, which is what lets the gateway and the API make access decisions without a database lookup. Section 7 covers how that is enforced.

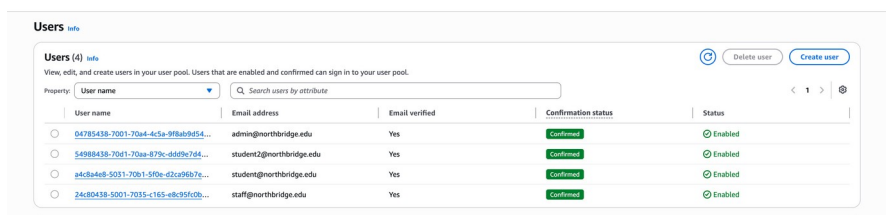


Figure 8: The Cognito user pool, which handles accounts, sign in and tokens.

Compute

One Go binary runs as a Lambda function on arm64 with 256 MB of memory and a 15 second timeout. Graviton costs about 20 percent less than x86 for the same work.

The same binary runs locally as an ordinary HTTP server, which made development fast and meant we could test handlers without deploying anything.

We also wrote a second function, the escalator, which raises the priority of service requests left unanswered past the deadline. It is in the repository but we have not deployed it yet, so staff escalate overdue requests by hand for now.

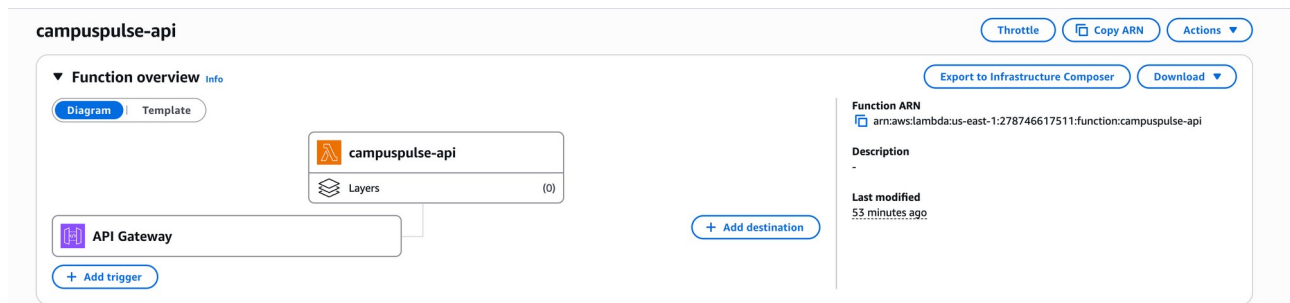


Figure 9: The API Lambda: a custom Go runtime on arm64, 256 MB, 15 second timeout.

Storage

Six DynamoDB tables, all on demand so we pay per request rather than for reserved capacity. Table names share the campuspulse- prefix, which let us scope IAM policies tightly.

Table	Key	What it holds	TTL
campuspulse-events	event_id	Raw sensor readings	30 days
campuspulse-rooms	room_id	Room records and current status	none
campuspulse-alerts	alert_id	Problems detected by the rules	none
campuspulse-service-requests	request_id	Student requests and their state	none
campuspulse-aggregates	pk + sk	Pre-computed counters for the dashboard	48 hours
campuspulse-settings	id	Admin threshold configuration	none

The events table has a time to live of 30 days. Raw readings pile up fastest and are the least useful once they have been aggregated, so DynamoDB deletes them for free rather than us paying to store them.

Tables (6) info											
Find tables				Filter by tag Any tag key		Filter by tag value Any tag value		Last updated October 3, 2026, 16:47 (UTC+2:00) Actions Delete Create table			
☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size
☐	campuspulse-aggregates	Active	pk (S)	sk (S)	0	1 Region	Off	☆	On-demand	On-demand	1.6 kilobytes
☐	campuspulse-alerts	Active	alert_id (S)	-	2	1 Region	Off	☆	On-demand	On-demand	65 kilobytes
☐	campuspulse-events	Active	event_id (S)	-	2	0	Off	☆	On-demand	On-demand	9 megabytes
☐	campuspulse-rooms	Active	room_id (S)	-	0	1 Region	Off	☆	On-demand	On-demand	7.3 kilobytes
☐	campuspulse-service-requests	Active	request_id (S)	-	3	1 Region	Off	☆	On-demand	On-demand	18.7 kilobytes
☐	campuspulse-settings	Active	id (S)	-	0	1 Region	Off	☆	On-demand	On-demand	457 bytes

Figure 10: The six DynamoDB tables that make up the CampusPulse data layer.

campuspulse-aggregates ☆ campuspulse-alerts ☆ campuspulse-events ☆ campuspulse-rooms ☆ campuspulse-service-requests ☆ campuspulse-settings ☆											
Filters - optional Run Reset											
Stopped • Items returned: 50 • Items scanned: 50 • Efficiency: 100% • RCUs consumed: 2 Retrieve next page											
Table: campuspulse-events - Items returned (50) Scan started on October 03, 2026, 16:48:31 Actions Create item											
☐	event_id (String)	building	day	device_id	event_type	expires_at (TTL)	received_at	room	room_id	sev	
☐	evt-c331560ff4ad944...	StudentHu...	2026-09-18	env-studen...	humidity	1792332000	2026-09-19T...	D203	studenthub...	nor	
☐	evt-d199d2c073451f...	Science-C	2026-09-18	env-science...	temperature	1792352700	2026-09-19T...	C201	science-c-c...	nor	
☐	evt-f358cdd7ff6b4fa...	Engineering-B	2026-09-19	occ-engine...	occupancy	1792383300	2026-09-19T...	B102	engineering...	nor	
☐	evt-52f0363596229b...	Science-C	2026-09-18	occ-science...	occupancy	1792340100	2026-09-19T...	C101	science-c-c...	nor	
☐	evt-4d4df53d...	Engineering-B	2026-09-18	env-engine...	humidity	1792366200	2026-09-19T...	B201	engineering...	nor	
☐	evt-9ed3aecd62ec1d...	Science-C	2026-09-19	env-science...	temperature	1792379700	2026-09-19T...	C102	science-c-c...	nor	
☐	evt-5b1e3bc1cba53f3...	Engineering-B	2026-09-19	env-engine...	humidity	1792413900	2026-09-19T...	B203	engineering...	nor	

Figure 11: Sensor events as stored in DynamoDB, matching the data model in Appendix A.

Networking

The API Lambda runs in private subnets across two availability zones. There is no internet gateway and no NAT gateway, so the code physically cannot reach the internet.

It reaches DynamoDB through a gateway VPC endpoint, which keeps the traffic on the AWS network and is free. A NAT gateway would have cost around 35 dollars a month on its own, which would have blown the budget by itself.

Your VPCs

VPCs

VPC encryption controls

Your VPCs (3) [info](#)

Last updated
1 minute ago

Actions

Create VPC

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR	DHCP option set
☐	campuspulse-vpc	vpc-07f5719b4808a5697	Available	-	-	Off	10.0.0.0/16	-	dopt-0c5d0cfd1
☐	MyVPC1	vpc-0ae4b0b3cc1e97d28	Available	-	-	Off	120.0.0.0/16	-	dopt-0c5d0cfd1
☐	-	vpc-04cd5ae955b84c1ae	Available	-	-	Off	172.31.0.0/16	-	dopt-0c5d0cfd1

Figure 12: The VPC: two private subnets, no internet gateway, no NAT gateway.

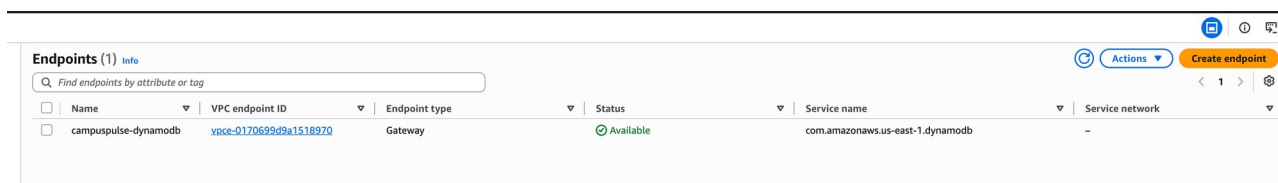


Figure 13: The DynamoDB gateway endpoint, which keeps database traffic off the public internet.

3. Technology Stack

We picked tools we could justify on cost, and that fit a serverless design.

Layer	Choice	Why
Compute	AWS Lambda, Go 1.x on arm64	Free Tier covers 1M requests a month, and Go starts cold in milliseconds
API	API Gateway HTTP API	Cheaper than REST API, built in JWT authorizer and CORS
Storage	DynamoDB on demand	25 GB always free, no capacity planning, TTL deletes old data free
Auth	Amazon Cognito, Lite tier	Managed tokens, hosted login page, groups map to our roles
Frontend	React, TanStack Router, Vite, Tailwind	Static build, so hosting costs almost nothing
Hosting	S3 and CloudFront with OAC	Private bucket, HTTPS, global cache
Monitoring	CloudWatch dashboard, alarms, logs, SNS	Free Tier covers 10 alarms and 5 GB of logs
IaC	Terraform 1.11 with S3 backend	The whole account rebuilds from code
CI/CD	GitHub Actions with OIDC	Free for public repositories, no stored AWS keys
Simulator	Go CLI	Shares the models and validation with the API

3.1 Why Go

Go compiles to a single static binary with no runtime to ship, so the Lambda package is small and cold starts stay under a tenth of a second. The same code runs locally, in Docker, and in Lambda with no changes.

3.2 Why not EC2

An EC2 instance would have been simpler to explain, but a t2.micro runs all month whether anyone uses it or not, and the Free Tier only covers 12 months. Lambda bills per request, so an idle campus at 3am costs nothing.

4. Implementation Process

4.1 How we built it, in order

We built the infrastructure in numbered steps, and the Terraform files still carry those step numbers in their comments. The order mattered, because each layer depends on the one before it.

Step	What we added	File
1	Budget alert, before anything could cost money	budget.tf
2	DynamoDB tables	dynamodb.tf
3	IAM roles and least privilege policies	iam.tf
4	VPC, subnets, endpoint, security groups	vpc.tf
5	Cognito user pool, groups and app client	cognito.tf
6	Lambda functions	lambda.tf
7	API Gateway, routes, authorizer, throttling	api_gateway.tf
8	S3 and CloudFront for the dashboard	frontend.tf
9	CloudWatch dashboard and alarms	monitoring.tf
10	Cross region replication for disaster recovery	replication.tf

4.2 Deployment workflow

A full deployment is three commands. The Lambda binary gets built first, because Terraform zips the compiled file and uses its hash to decide whether to publish a new version.

```
sh infra/build-lambda.sh
cd infra && terraform apply
sh infra/deploy-dashboard.sh
```

In practice we rarely run these by hand any more. Pushing to main does all three through GitHub Actions, which is covered in section 6.

```

➤ infra git:(main) ✗ ls
➤ infra git:(main) ✗ terraform plan
random_password.device_key: Refreshing state... [id=none]
data.archive_file.backend: Reading...
random_string.arth_domain: Refreshing state... [id=jac55o]
data.archive_file.backend: Read complete after 0s [id=2148afec12aed4d692003d9b00da1224d1c3f6df]
data.aws_iam_policy_document.replication_assume: Reading...
data.aws_caller_identity.current: Reading...
data.aws_availability_zones.available: Reading...
data.aws_iam_policy_document.lambda_assume: Reading...
aws_cloudfront_origin_access_control.dashboard: Refreshing state... [id=E18XXBE092G6ZM]
aws_cloudwatch_log_group.api: Refreshing state... [id=/aws/lambda/campuspulse-api]
aws_cloudwatch_log_group.api.access: Refreshing state... [id=/aws/apigateway/campuspulse-api]
aws_vpc.main: Refreshing state... [id=vpc-07f5719b4080a5697]
aws_sns_topic.alarms: Refreshing state... [id=arn:aws:sns:us-east-1:278746617511:campuspulse-alarms]
data.aws_iam_policy_document.lambda_assume: Read complete after 0s [id=2690255455]
data.aws_iam_policy_document.replication_assume: Read complete after 0s [id=1765856792]
aws_budgets.budget.monthly: Refreshing state... [id=278746617511:campuspulse-monthly]
aws_dynamodb_table.this["aggregate"]: Refreshing state... [id=campuspulse-aggregates]
aws_cognito_user_pool.main: Refreshing state... [id=us-east-1_2NnTAzW7c]
data.aws_caller_identity.current: Read complete after 1s [id=278746617511]
aws_dynamodb_table.this["alerts"]: Refreshing state... [id=campuspulse-alerts]
aws_dynamodb_table.this["rooms"]: Refreshing state... [id=campuspulse-rooms]
data.aws_availability_zones.available: Read complete after 1s [id=us-east-1]
aws_dynamodb_table.this["service-requests"]: Refreshing state... [id=campuspulse-service-requests]
aws_dynamodb_table.this["events"]: Refreshing state... [id=campuspulse-events]
aws_iam_role.replication: Refreshing state... [id=campuspulse-s3-replication]
aws_dynamodb_table.this["settings"]: Refreshing state... [id=campuspulse-settings]
aws_iam_role.api: Refreshing state... [id=campuspulse-api-lambda]
aws_s3_bucket.dashboard: Refreshing state... [id=campuspulse-dashboard-278746617511]
aws_s3_bucket.replica["dashboard"]: Refreshing state... [id=campuspulse-dashboard-278746617511-eu-west-3]
aws_sns_topic_subscription.alarms_email: Refreshing state... [id=arn:aws:sns:us-east-1:278746617511:campuspulse-alarms:e49b6037-a781-4fe6-99cf-5df2f7c764dd]
aws_s3_bucket.replica["tfstate"]: Refreshing state... [id=campuspulse-tfstate-278746617511-eu-west-3]
aws_cognito_user_pool_domain.main: Refreshing state... [id=campuspulse-jac55o]

```

Figure 14: A successful terraform apply, showing the stack outputs.

4.3 Configuration decisions worth noting

- **One binary, two modes:** the Lambda reads TABLE_PREFIX, AUTH_MODE and COGNITO_CLIENT_ID from its environment, so the same binary runs locally against Docker DynamoDB and in AWS against the real thing.
- **Device key:** sensors authenticate with a 40 character key that Terraform generates and never prints. It is marked sensitive in the outputs.
- **Timestamps:** all timestamps use a fixed width UTC format, so they sort correctly as plain strings in DynamoDB.
- **Thresholds in the database:** an admin can change occupancy, temperature, humidity and energy thresholds at runtime through PUT /thresholds, with no redeploy.

4.4 Problems we hit

CORS preflight requests were rejected

Browsers never send the Authorization header on preflight OPTIONS requests, so the \$default route's token check rejected them and every dashboard call failed. The fix was an explicit public route for OPTIONS /{proxy+}, which lets preflight through before the authorizer sees it.

The Lambda could not reach Cognito

Our first design had the Lambda verifying tokens itself, which meant calling Cognito over the internet from a subnet with no route to it. Rather than pay for a NAT gateway, we moved token verification to the API Gateway authorizer and had the Lambda trust the claims the gateway passes through. Cheaper and faster.

OIDC trust policy rejected the deploy role

GitHub rolled out immutable OIDC subjects, which changed the format of the sub claim, so the first deploy failed with an AssumeRoleWithWebIdentity error. The trust policy needed updating to match the new subject format.

The single page app returned 403 on refresh

Refreshing on /operations asked S3 for a file that does not exist. CloudFront custom error responses now map 403 and 404 to /index.html with a 200 status.

5. Infrastructure as Code with Terraform

We defined the whole platform in Terraform, so that the account can be destroyed and rebuilt from the repository. Nothing here was created by clicking in the console.

5.1 How the code is organised

File	What it defines
providers.tf	AWS provider, region, S3 backend for shared state
variables.tf	Project name, region, budget, emails, replica region
budget.tf	Monthly budget with actual and forecast alerts
dynamodb.tf	All six tables, generated from one map with for_each
iam.tf	Execution roles, least privilege policies, OIDC deploy role
vpc.tf	VPC, private subnets, DynamoDB endpoint, security groups
cognito.tf	User pool, groups, app client, hosted domain, demo users
lambda.tf	API function, its log group and environment
api_gateway.tf	HTTP API, routes, JWT authorizer, throttling, access logs
frontend.tf	Dashboard bucket, CloudFront, OAC, failover origin group
monitoring.tf	CloudWatch dashboard, seven alarms, SNS topic
replication.tf	Cross region buckets and replication rules
outputs.tf	URLs and IDs that the deploy scripts read

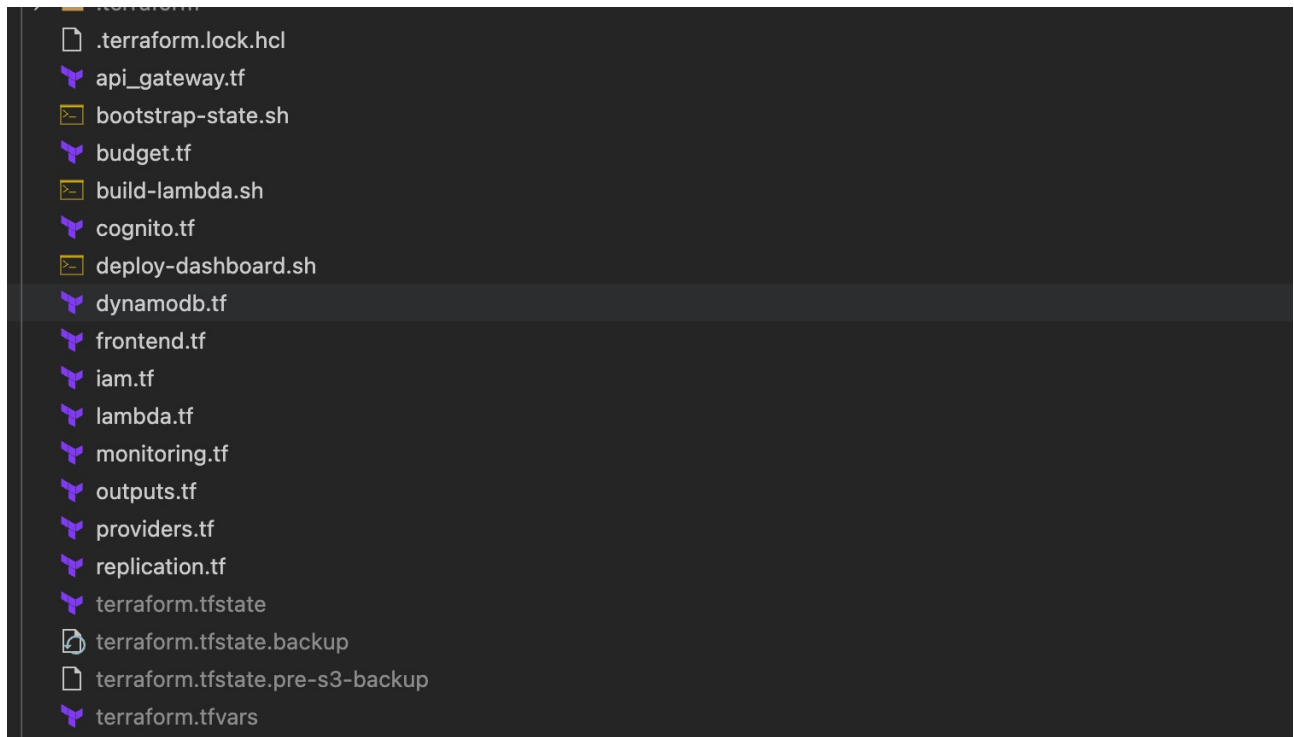


Figure 15: The Terraform codebase. Thirteen files describe the entire platform.

5.2 Patterns we used

Remote state in S3

State lives in a versioned, encrypted S3 bucket with a lock file, so the team and the CI/CD pipeline share one source of truth and two applies cannot run at once. State contains secrets, so the bucket denies non HTTPS requests and blocks all public access.

Secrets never touch the repository

terraform.tfvars is in .gitignore. The device key is generated by Terraform. The demo password and alert email arrive as TF_VAR environment variables from GitHub secrets.

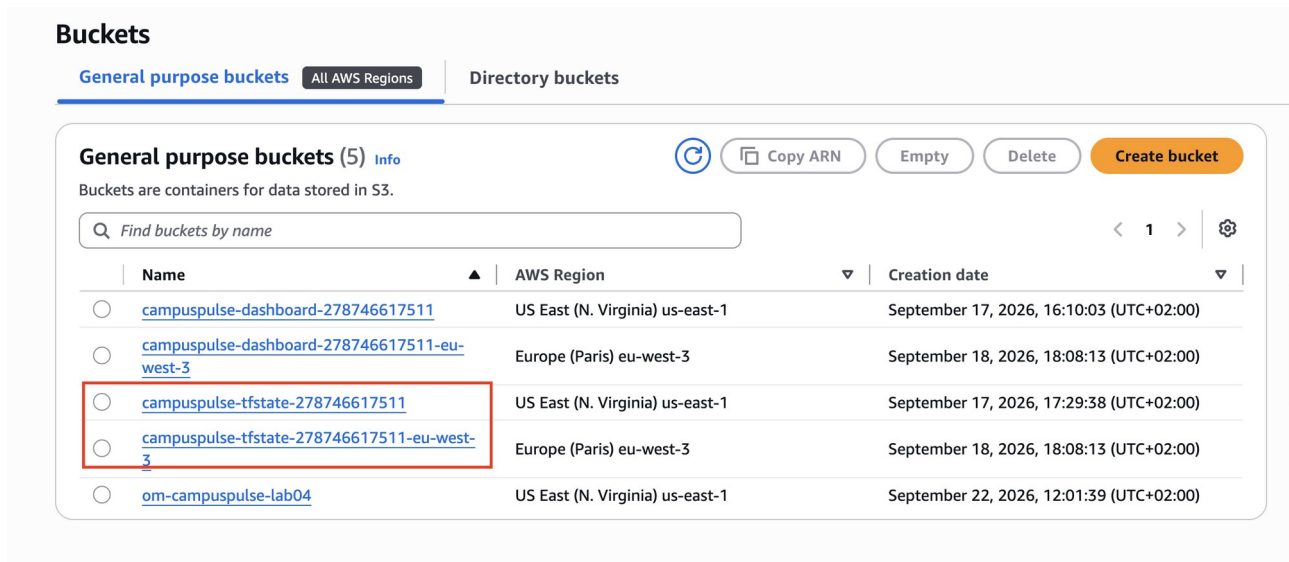


Figure 16: Terraform state: versioned, encrypted and private.

6. CI/CD with GitHub Actions

We set up CI/CD with GitHub Actions, so that tests run on every pull request and a push to main deploys the whole platform on its own. We did it with two workflows, one that guards pull requests and one that deploys, and we did it without storing an AWS access key anywhere.

6.1 Pull request checks

Every pull request runs three jobs in parallel, with no AWS permissions at all. A failing job shows as a red check on the pull request, so problems are visible before anything is merged.

Job	What it runs
Backend (Go)	go vet ./... and go test ./...
Infra (Terraform)	terraform fmt -check, terraform init -backend=false, terraform validate
Dashboard (Bun)	bun install and bun run build

6.2 Deployment

Pushing to main triggers the deploy workflow, which runs five steps in order:

- 1. **Authenticate:** assume the AWS role through OIDC and receive short lived credentials.
- 2. **Build:** compile the Go binary for arm64 Linux.
- 3. **Plan and apply:** terraform plan writes a plan file, then terraform apply applies that exact file.
- 4. **Deploy the frontend:** build the dashboard, sync it to S3, invalidate the CloudFront cache.
- 5. **Verify:** call /health with retries, so a broken deploy fails the run loudly.

A concurrency group stops two deployments running at once, which matters because Terraform state is shared.

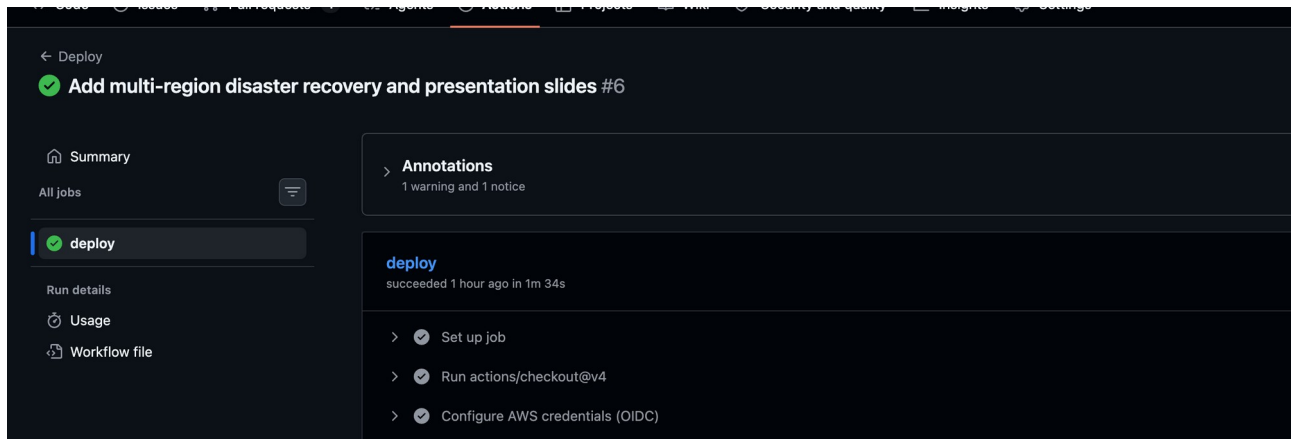


Figure 17: Deployment history. Every push to main redeploys the whole platform.

6.3 Keyless authentication

This is the part we are most pleased with. There are no AWS access keys in GitHub, in the repository or on anyone's laptop.

An IAM OpenID Connect provider trusts GitHub's token issuer, and the deploy role trusts only the main branch of this one repository. GitHub mints a short lived token per run, AWS exchanges it for temporary credentials, and they expire when the run ends.

The role uses a custom campuspulse-deploy-policy rather than AdministratorAccess. S3 access is limited to two buckets, and Lambda, DynamoDB, IAM and SNS are limited to resources named campuspulse-.*.

7. Security Design

7.1 Threats we designed against

Threat	What we did about it
Anyone reading campus data	Every data route needs a Cognito token, checked at the gateway
A student acting as staff	Roles come from Cognito groups inside the signed token, never from the request body
Fake sensor data	Sensor routes require a 40 character device key generated by Terraform
Credential stuffing	No public sign up, admin created accounts only, 4xx spike alarm
Request floods and cost attacks	Gateway throttling at 20 requests per second, burst 50
Stolen AWS keys	There are none. Deployment uses short lived OIDC credentials
Data theft from storage	Private buckets, encryption at rest everywhere, HTTPS only

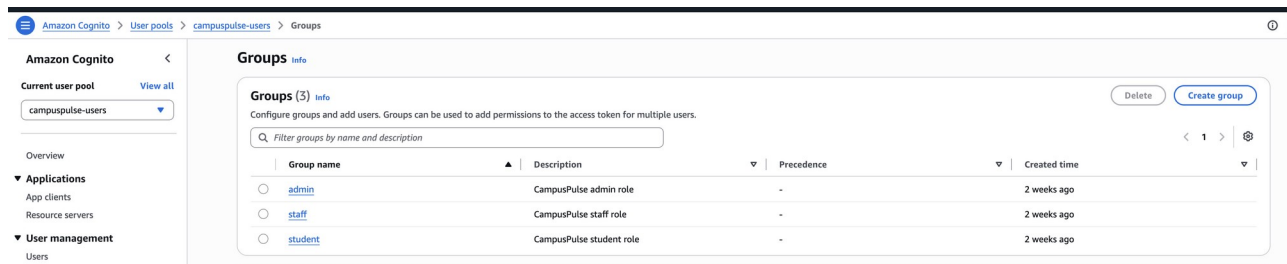
A compromised Lambda	No internet route out, and IAM limited to campuspulse-* tables
----------------------	--

7.2 Identity and access

Cognito holds the users. Three groups map to the three roles in the brief: student, staff and admin. The group appears in the token as cognito:groups, and the API reads it from there, so a user cannot promote themselves by editing a request.

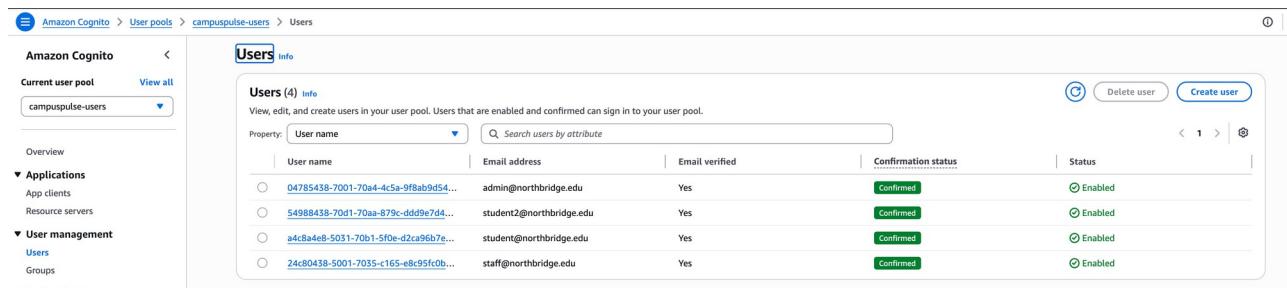
Access is enforced per route. Students can raise service requests, staff can see campus data and manage alerts, and only admins can change thresholds.

Role	Can do
student	See rooms, raise and track their own service requests
staff	Everything above, plus events, alerts, stats and managing requests
admin	Everything above, plus changing the threshold rules



The screenshot shows the Amazon Cognito console interface. On the left, there's a navigation menu with 'Overview', 'Applications', and 'User management'. The 'User management' section is expanded, showing 'Users' and 'Groups'. The 'Groups' page is active, displaying a list of three groups: 'admin', 'staff', and 'student'. Each group has a description, a precedence value, and a creation time of '2 weeks ago'. The 'admin' group is described as 'CampusPulse admin role', 'staff' as 'CampusPulse staff role', and 'student' as 'CampusPulse student role'.

Figure 18: The three Cognito groups that carry our role model.



The screenshot shows the Amazon Cognito console interface for the 'Users' page. The left navigation menu is the same as in Figure 18. The 'Users' page displays a list of four users. Each user has a unique ID, an email address, a confirmed status (indicated by a green 'Confirmed' label), and an enabled status (indicated by a green 'Enabled' label). The users are: 'admin@northbridge.edu', 'student2@northbridge.edu', 'student@northbridge.edu', and 'staff@northbridge.edu'.

Figure 19: Accounts are created by an administrator. Public sign up is disabled.

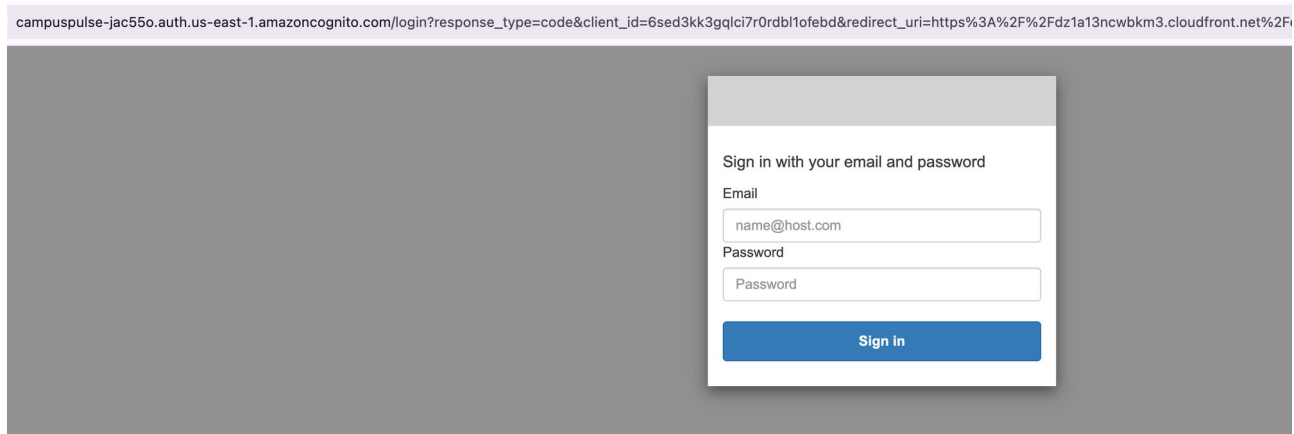


Figure 20: The hosted sign in page. Passwords never reach our own code.

7.3 Secure networking

The Lambda functions sit in private subnets with no internet gateway and no NAT gateway. The security group allows no inbound traffic at all and only HTTPS outbound to the DynamoDB prefix list.

We also stripped the allow all rules from the VPC's default security group, so anything accidentally launched into the VPC gets no network access.

☐	Name	Security group ID	Security group name	VPC ID	Description	Owner
☐	campuspulse-lambda	sg-06dc0a915bbfc77a2	campuspulse-lambda	vpc-07f5719b4808a5697_l2	CampusPulse Lambda functions: no inb...	278746617511
☐	campuspulse-default...	sg-0b97b5cb36e71adb6	default	vpc-07f5719b4808a5697_l2	default VPC security group	278746617511

Figure 21: The Lambda security group: nothing in, HTTPS to DynamoDB only.

7.4 Encryption and secrets

- **In transit:** TLS everywhere. CloudFront redirects HTTP to HTTPS, and the state bucket denies plain HTTP outright.
- **At rest:** DynamoDB, S3 and Lambda environment variables are all encrypted with AWS managed keys.
- **Secrets:** the device key is generated by Terraform and marked sensitive. Nothing sensitive is committed.

7.5 Least privilege in IAM

The Lambda execution role can touch only the campuspulse-* tables and write its own logs. It cannot read other tables, create resources or talk to other services.

Roles (9) [Info](#)

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Q Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForAPIGateway	AWS Service: ops.apigateway (Service-Linked Role)	-
☐	AWSServiceRoleForDynamoDBReplication	AWS Service: replication.dynamodb (Service-Linked Role)	4 minutes ago
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	24 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	16 days ago
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
☐	campuspulse-api-lambda	AWS Service: lambda	5 minutes ago
☐	campuspulse-s3-replication	AWS Service: s3	32 minutes ago
☐	DemoRoleForEC2	AWS Service: ec2	74 days ago
☐	github-actions-campuspulse	Identity Provider: arn:aws:iam::278746617511:oidc-provider/token.actions.githubuse	1 hour ago

Figure 22: The execution role is scoped to our tables, not to DynamoDB as a whole.

8. Scalability Strategy

8.1 What scales on its own

Almost everything, because almost everything is managed. Lambda adds concurrent executions as requests arrive. DynamoDB on demand absorbs traffic spikes without capacity planning. CloudFront caches the dashboard at edge locations worldwide.

The API holds no state in memory, so any invocation can serve any request. That is what makes horizontal scaling free.

8.2 Where it would break first

Being honest about limits matters more than claiming infinite scale.

Pressure point	Current limit	What we would do
API Gateway throttling	20 requests per second, burst 50	Raise the limit, it is one Terraform line
Lambda concurrency	1000 account default	Request an increase, or add reserved concurrency
Per request writes	One event, several writes	Batch writes, or buffer through SQS
Hot partitions	Busy rooms write to the same key	Add a shard suffix to the partition key
Dashboard queries	Scans on bigger tables	More GSIs, or pre-aggregate further

8.3 Growing from 50 rooms to 5000

When the campus grows, the platform gets pressure from two directions, and they grow for different reasons. Data coming in grows with the number of sensors. Data going out grows with the number of

people looking at the dashboard. Ten thousand sensors with one person watching is heavy in one direction and trivial in the other, so we looked at them separately.

Data coming in. Say the campus grows from 50 rooms to 5000, each reporting every 30 seconds. That is about 170 readings a second. It sounds like a lot, but the simulator already sends readings in batches of up to 25 through POST /events/batch, so it arrives as about 7 requests a second.

That is inside our current limit of 20 requests a second, so the first thing to change is not the limit, it is what happens when a burst arrives. Today anything above the limit is simply rejected. The SQS queue in section 2.2 would hold those events instead and let the Lambda work through them at its own pace.

Data going out. This grows with the number of people watching the dashboard, not with the number of sensors, and it travels the same path in reverse: the browser calls API Gateway, the Lambda runs, and the Lambda reads DynamoDB.

What saves us there is which tables it reads. For the campus overview the Lambda never trawls through raw events. It reads the rooms table, where each room's latest reading is already stored, plus open alerts, open requests, and the hourly energy totals we pre-compute in the aggregates table. Only a single room's detail page touches the events table, and only for that one room.

9. Cost Analysis

9.1 What we actually pay

We designed the platform to run inside the Free Tier, and it does. The table below shows our measured or expected monthly usage against each allowance.

Service	Our usage	Free Tier allowance	Cost
Lambda	Well under 100k requests	1M requests, 400k GB-seconds	0.00 USD
API Gateway	Under 100k calls	1M HTTP API calls (12 months)	0.00 USD
DynamoDB	Under 1 GB, on demand	25 GB storage, 25 WCU and RCU	0.00 USD
S3	Around 1 MB of static files	5 GB standard storage	0.00 USD
CloudFront	Light demo traffic	1 TB out, 10M requests	0.00 USD
Cognito Lite	Under 20 users	10,000 monthly active users	0.00 USD
CloudWatch	7 alarms, 7 day log retention	10 alarms, 5 GB logs	0.00 USD
VPC endpoint	Gateway endpoint for DynamoDB	Gateway endpoints are free	0.00 USD
Cross region replication	Small objects, low volume	Not free, but cents	Under 1 USD
Total	Under 1 USD per month		

9.2 Decisions that kept it free

- **The big one:** no NAT gateway. This alone would have cost around 35 USD a month. We used a free gateway endpoint instead.
- **API choice:** HTTP API instead of REST API, roughly a third of the price.
- **Architecture:** arm64 Lambda, about 20 percent cheaper than x86.
- **Data expiry:** 30 day TTL on events and 48 hours on aggregates, so storage does not grow forever.
- **Logs:** 7 day log retention instead of the default forever.
- **Billing mode:** on demand DynamoDB, so idle tables cost nothing.
- **What we avoided:** no EC2, no RDS, no load balancer, no Kubernetes.

9.3 Billing alert

A monthly AWS Budget is defined in Terraform, in `budget.tf`. It emails at 80 percent of the limit based on actual spend, and again when AWS forecasts the month will go over. The budget existed before any resource was created, which is why it is step 1 in our build order.

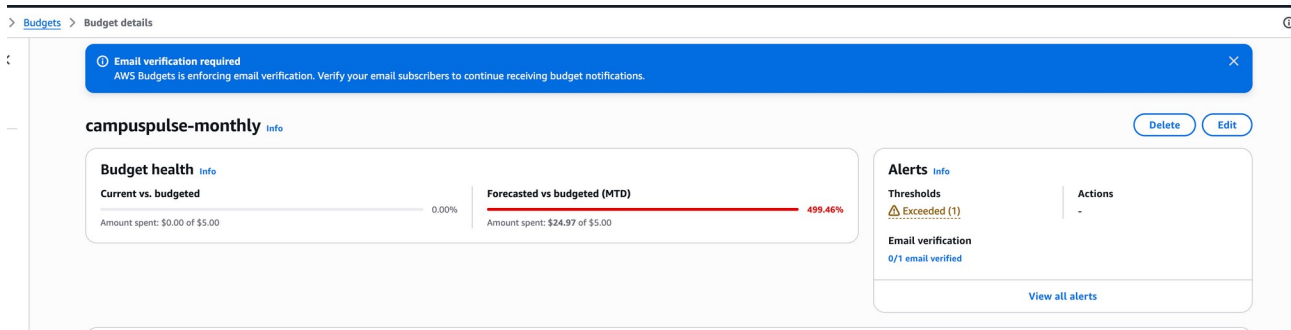


Figure 23: The monthly budget with alerts at 80 percent actual and 100 percent forecast.

Free Tier offers in use (5)							
Find service name							
Service	AWS Free Tier usage limit	Current usage	Forecasted usage	MTD actual usage %	MTD forecasted usage %		
AmazonCloudWatch	10.0 Alarms are always free per month as part of AWS Free Usage Tier (Global-CW:AlarmMonitorUsage)	1 Alarms	6 Alarms	5.70%	58.89%		
AmazonCloudWatch	3.0 Dashboards are always free per month as part of AWS Free Usage Tier (Global-DashboardsUsageHour-Basic)	0 Dashboards	1 Dashboards	4.75%	49.07%		
Amazon DynamoDB	25.0 GB-Mo are always free per month as part of AWS Free Usage Tier (TimedStorage-ByteHrs)	0 GB-Mo	0 GB-Mo	0.00%	0.03%		

Figure 24: Free Tier usage. Every service is well inside its allowance.

9.4 The one paid resource

Cross region replication is not in the Free Tier. It costs a few cents a month at our data volume, and we judged it worth that, because the brief requires a disaster recovery plan and Terraform state is the one thing we cannot rebuild if it is lost.

10. Observability

10.1 The dashboard

We built a CloudWatch dashboard called CampusPulse, so that API traffic, Lambda duration and errors, DynamoDB activity and our own business metrics sit side by side and a problem can be spotted without digging through logs.

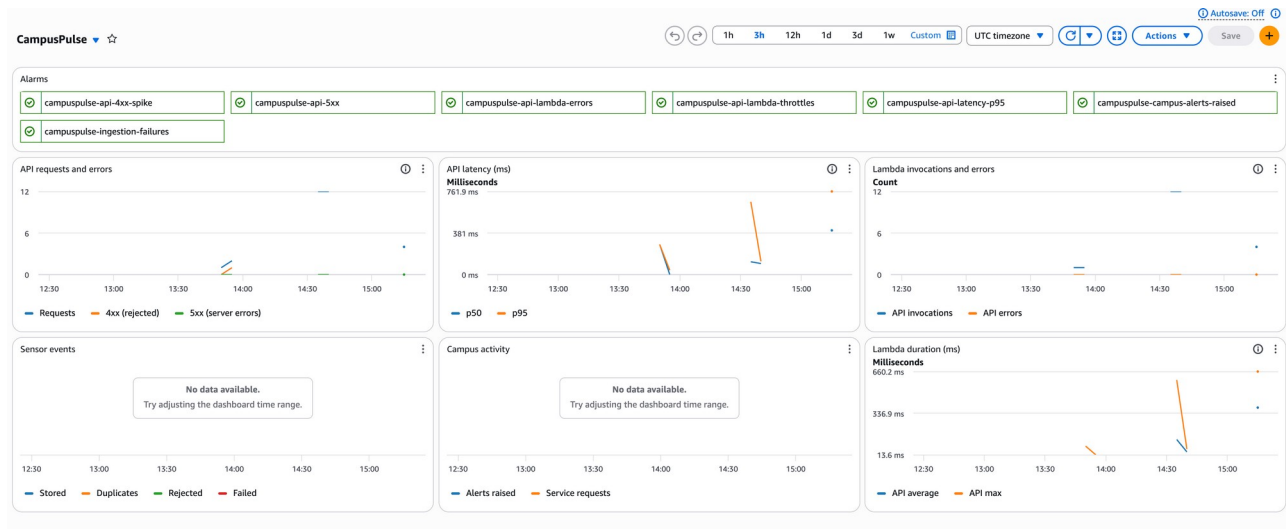


Figure 25: The CloudWatch dashboard: infrastructure and campus metrics in one view.

10.2 Alarms

Seven alarms publish to an SNS topic that emails the team. Six come from a map in Terraform, plus a latency alarm.

Alarm	Fires when
api-lambda-errors	The API Lambda crashed or timed out
api-lambda-throttles	Lambda refused requests because too many ran at once
api-5xx	Five or more server errors in five minutes
api-4xx-spike	Fifty or more rejected requests in five minutes, a possible attack
api-latency-p95	95th percentile latency goes above 3 seconds
ingestion-failures	A sensor event could not be stored
campus-alerts-raised	The platform detected a real campus problem

The last one is worth calling out. It is not an infrastructure alarm, it is a business alarm. It fires when the platform does its job and detects overcrowding or overheating.

Alarms							
Mute rules							
Alarms (8)							
Filter alarms Quick filters Clear selection Create composite alarm Actions Create alarm							
☐	Name	State	Actions	Actions muted	Last state update (UTC)	Conditions	
☐	BillingAlarmUSD500	OK	Actions enabled	-	2026-09-22 16:51:43	EstimatedCharges > 500 for 1 datapoints within 6 hours	
☐	campuspulse-campus-alerts-raised	OK	Actions enabled Warning	-	2026-09-19 14:04:24	AlertsRaised >= 1 for 1 datapoints within 5 minutes	
☐	campuspulse-api-latency-p95	OK	Actions enabled Warning	-	2026-09-17 14:26:47	Latency > 3000 for 3 datapoints within 15 minutes	
☐	campuspulse-ingestion-failures	OK	Actions enabled Warning	-	2026-09-17 14:26:42	EventsFailed >= 1 for 1 datapoints within 5 minutes	
☐	campuspulse-api-lambda-throttles	OK	Actions enabled Warning	-	2026-09-17 14:26:41	Throttles >= 1 for 1 datapoints within 5 minutes	
☐	campuspulse-api-lambda-errors	OK	Actions enabled Warning	-	2026-09-17 14:26:37	Errors >= 1 for 1 datapoints within 5 minutes	
☐	campuspulse-api-5xx	OK	Actions enabled Warning	-	2026-09-17 14:26:30	5xx >= 5 for 1 datapoints within 5 minutes	
☐	campuspulse-api-4xx-spike	OK	Actions enabled Warning	-	2026-09-17 14:26:09	4xx >= 50 for 1 datapoints within 5 minutes	

Figure 26: All seven alarms, with their current state.

10.3 Logs

API Gateway writes a structured JSON access log for every request, including request id, status, latency and any authorizer error. The Lambda writes its own application logs. Both groups keep 7 days, which stays inside the Free Tier.

Every response carries an x-request-id header, so a user report can be traced to one log line.

CloudWatch > Log management									
Log groups									
Data sources - new									
Summary									
Get-set-go with unified data store									
Log groups (2)									
By default, we only load up to 10,000 log groups.									
Filter log groups or try pattern search ☒ Exact match									
☐	Log group	Log class	Anomaly d...	Deletion protection	Bearer token auth...	Data protection	Sensitive data co...	Reter	
☐	/aws/apigateway/campuspulse-api	Standard	Configure	Off	Off	-	-	1 we	
☐	/aws/lambda/campuspulse-api	Standard	Configure	Off	Off	-	-	1 we	

Figure 27: Structured access logs. Each entry carries status, latency and request id.

11. Disaster Recovery Strategy

11.1 What we protect, and how

We ranked our data by how painful losing it would be, then protected it accordingly. Not everything deserves the same treatment, and replicating everything would cost money for no benefit.

Asset	Protection	Why
Terraform state	Versioned S3, replicated to eu-west-3	The one thing we cannot rebuild
Rooms, alerts, requests,	DynamoDB global tables in eu-west-3,	Operational data worth keeping

settings, aggregates	PITR on	
Raw events	Single region, 30 day TTL	High volume, regenerable, expires anyway
Dashboard files	Versioned S3, replicated, CloudFront failover	Rebuildable, but failover is nearly free
Application code	GitHub	Rebuilds everything else

11.2 Recovery objectives

Scenario	RPO	RTO	How we recover
Bad deployment	0	Under 10 minutes	Revert the commit, the pipeline redeploys
Accidental data deletion	5 minutes	Under 1 hour	DynamoDB point in time recovery
Dashboard broken by a bad upload	0	Minutes	Restore the previous S3 version
Terraform state corrupted	0	Minutes	Restore the previous object version
Full region outage	Seconds	1 to 2 hours	Dashboard fails over automatically, redeploy compute in eu-west-3

11.3 What is automatic and what is not

CloudFront failover is automatic. If the primary bucket returns a server error, CloudFront retries the same request against the eu-west-3 replica, and visitors see nothing.

Data replication is automatic and continuous, through DynamoDB global tables and S3 replication rules.

Compute failover is manual, and we chose that deliberately. Running a second Lambda, API Gateway and Cognito pool in another region costs money and adds complexity for a student project. Because everything is in Terraform, bringing the stack up in eu-west-3 is a variable change and an apply, which is where the 1 to 2 hour RTO comes from.

Buckets

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (5) Info

Refresh

Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

< 1 > ⚙

	Name	AWS Region	Creation date
☐	campuspulse-dashboard-278746617511	US East (N. Virginia) us-east-1	September 17, 2026, 16:10:03 (UTC+02:00)
☐	campuspulse-dashboard-278746617511-eu-west-3	Europe (Paris) eu-west-3	September 18, 2026, 18:08:13 (UTC+02:00)
☐	campuspulse-tfstate-278746617511	US East (N. Virginia) us-east-1	September 17, 2026, 17:29:38 (UTC+02:00)
☐	campuspulse-tfstate-278746617511-eu-west-3	Europe (Paris) eu-west-3	September 18, 2026, 18:08:13 (UTC+02:00)
☐	om-campuspulse-lab04	US East (N. Virginia) us-east-1	September 22, 2026, 12:01:39 (UTC+02:00)

Figure 28: Four buckets: two primaries in us-east-1 and two replicas in eu-west-3.

Replication rules (1)

Refresh

View details

Edit rule

Delete

Actions

Create replication rule

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. Replication also includes any annotations attached to your objects. [Learn more](#)

Replication now includes replicating any annotations attached to your objects, which might increase your costs. To understand the potential cost impact, review your replication rules and annotations usage.

Replication rule name	Status	Destination bucket	Destination Region	Priority	Scope	Storage class	Replica owner	Replication Time Control	KMS-encrypted objects (SSE-KMS or DSSE-KMS)	Replica modification sync
☐ to-replica-region	Enabled	s3://campuspulse-dashboard-278746617511-eu-west-3	Europe (Paris) eu-west-3	0	Entire bucket	Transition to Standard	Same as source	Disabled	Do not replicate	Disabled

View replication configuration

Figure 29: The replication rule that copies every object to the recovery region.

campuspulse-rooms

Last updated

October 3, 2026, 17:29 (UTC+2:00)

Refresh

Star

Actions

Explore table items

Settings

Indexes

Monitor

Global tables

Backups

Exports and streams

Permissions

Summary

Multi-Region consistency

Eventual

Current Region

United States (N. Virginia) us-east-1

Version

2019.11.21 (Current)

Replicas (2) Info

Last updated

October 3, 2026, 17:29 (UTC+2:00)

Refresh

Delete replica

Create replica

Replicas of this table in other AWS regions and/or accounts.

Replication Region	Status	Type	Capacity mode	Endpoint
☐ Europe (Paris) eu-west-3	Active	Replica	On-demand	dynamodb.eu-west-3.amazonaws.com
☐ United States (N. Virginia) us-east-1	Active	Current Region	On-demand	dynamodb.us-east-1.amazonaws.com

Figure 30: DynamoDB global tables: five of six tables have a live replica in eu-west-3.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size
campuspulse-aggregates	Active	pk (S)	sk (S)	0	1 Region	Off	☆	On-demand	On-demand	1.6 kilobytes
campuspulse-alerts	Active	alert_id (S)	-	2	1 Region	Off	☆	On-demand	On-demand	65 kilobytes
campuspulse-rooms	Active	room_id (S)	-	0	1 Region	Off	☆	On-demand	On-demand	7.3 kilobytes
campuspulse-service-requests	Active	request_id (S)	-	3	1 Region	Off	☆	On-demand	On-demand	18.8 kilobytes
campuspulse-settings	Active	id (S)	-	0	1 Region	Off	☆	On-demand	On-demand	460 bytes

Figure 31: The replica tables, live in Paris.

11.4 Backup policy

- **DynamoDB:** point in time recovery is on for every table, giving any second in the last 35 days.
- **S3:** versioning is on for both buckets, and old state versions expire after 90 days.
- **Code:** GitHub holds the full history, and the pipeline can rebuild the account from it.

12. Testing and Evaluation

12.1 Automated tests

We wrote 14 test functions covering the parts where bugs would be expensive: the rules engine, authentication, the HTTP API and the simulator. They run on every pull request.

Package	What it covers
internal/rules	Room status, event severity, energy evaluation, request priority, escalation, threshold validation, alert deduplication
internal/auth	Password hashing, token handling, Cognito claim parsing, ID token decoding
internal/api	Full HTTP surface, including role based access control
internal/sim	Simulator client behaviour

```

Campus Platform git:(main) x cd backend && go test ./... -count=1 -v 2>&1 | grep -E '^(== RUN |--- PASS|ok)' | head -40
== RUN TestAPI
ok      campuspulse/internal/api    0.450s
== RUN TestPasswords
--- PASS: TestPasswords (0.14s)
== RUN TestTokens
== RUN TestTokens/expired
== RUN TestTokens/wrong_secret
== RUN TestTokens/tampered_role
== RUN TestTokens/alg_none
--- PASS: TestTokens (0.00s)
== RUN TestCognitoLogin
== RUN TestUserFromGatewayClaims
== RUN TestUserFromGatewayClaims/single_group
== RUN TestUserFromGatewayClaims/most_privileged_group_wins
== RUN TestUserFromGatewayClaims/comma_separated
== RUN TestUserFromGatewayClaims/no_group
== RUN TestUserFromGatewayClaims/unknown_group
== RUN TestUserFromGatewayClaims/access_token
== RUN TestUserFromGatewayClaims/no_subject
--- PASS: TestUserFromGatewayClaims (0.00s)
== RUN TestDecodeIDToken
--- PASS: TestDecodeIDToken (0.00s)
ok      campuspulse/internal/auth    0.849s
== RUN TestRoomStatus
--- PASS: TestRoomStatus (0.00s)
== RUN TestEvaluateEvent
== RUN TestEvaluateEvent/occupancy_under_capacity
== RUN TestEvaluateEvent/occupancy_slightly_over
== RUN TestEvaluateEvent/occupancy_for_over
== RUN TestEvaluateEvent/comfortable_temperature
== RUN TestEvaluateEvent/warm_room
== RUN TestEvaluateEvent/overheating
== RUN TestEvaluateEvent/cold_room

```

Figure 32: The full backend test suite passing.

13. Lessons Learned

13.1 What worked

- **Infrastructure as code:** writing every resource in Terraform from day one meant we could destroy and rebuild without fear, and the report practically wrote itself from the code comments.
- **Budget first:** the budget alert was step 1, before any resource existed. We never had a cost surprise.
- **Checking tokens at the edge:** moving token verification to the API Gateway authorizer removed the need for internet access from the Lambda, which removed the need for a NAT gateway, which saved the budget.
- **The simulator:** building the simulator early gave us realistic data for the whole project, instead of testing against three hand typed records.

13.2 Trade offs we made

Decision	What we gained	What it cost
Serverless instead of EC2	Near zero idle cost, no patching	Cold starts, and Lambda specific debugging
No NAT gateway	About 35 USD a month saved	The Lambda cannot call outside services
Manual compute failover	No duplicate running costs	1 to 2 hour RTO instead of minutes
DynamoDB instead of RDS	Free Tier, no capacity planning	No joins, so we pre-aggregate instead
Events not replicated	Lower replication cost	30 days of raw events would be lost in a region failure

13.3 Limitations

- **One device key:** real sensors would need a device registry and per device keys, not one shared key.
- **No real time push:** dashboards poll the API. WebSockets or Server Sent Events would be smoother.
- **Rules, not models:** thresholds are fixed numbers. There is no learning of what normal looks like per room.
- **Escalation is manual:** the escalator function is written but not deployed, so overdue requests are escalated by staff rather than automatically.

13.4 What we would do next

- **Add a queue:** buffer ingestion through SQS, so traffic spikes queue instead of throttling.
- **Anomaly detection:** forecast occupancy and energy from the history we already store.

- **Device identity:** a per device registry with individual keys and revocation.
- **Live updates:** push alerts to the dashboard instead of polling for them.
- **Staging:** a staging environment, which is a workspace change in Terraform.

14. Conclusion

CampusPulse does what the brief asked for. It ingests simulated campus data continuously, processes it into room status and alerts, stores it durably across two regions, exposes it through a secured API and a live dashboard, and monitors itself.

It is deployed, it is running, and it costs under one dollar a month. Everything except cross region replication sits inside the AWS Free Tier.

The part we would defend most strongly is that nothing was clicked into existence. The whole platform is code, the pipeline deploys it without stored credentials, and a reviewer can read the repository and know exactly what is running in the account.

15. References and Reused Sources

As required by section 16 of the brief, everything we reused is listed here.

Source	Used for	Where
AWS documentation	Service configuration and Free Tier limits	Throughout
Terraform AWS provider docs	Resource definitions	infra/
aws-actions/configure-aws-credentials	OIDC login in the pipeline	deploy.yml
hashicorp/setup-terraform	Terraform in CI	ci.yml, deploy.yml
oven-sh/setup-bun	Dashboard build in CI	ci.yml, deploy.yml
aws-lambda-go	Lambda runtime adapter for Go	backend/cmd/lambda
AWS SDK for Go v2	DynamoDB and Cognito clients	backend/internal
TanStack Router, Vite, Tailwind	Dashboard framework and styling	campus-pulse-dashboard/
Lovable	First draft of the dashboard UI, then edited by us	campus-pulse-dashboard/
Claude (Anthropic)	Code review, documentation drafting, report preparation	Throughout, reviewed by the team

Appendix A: Data Model

Sensor events follow the format in Appendix A of the brief, with our own additions for traceability.

```
{
  "event_id": "evt-2026-0001",
  "building": "Library-A",
  "room": "A203",
  "event_type": "occupancy",
  "value": 42,
  "unit": "people",
  "severity": "normal",
  "timestamp": "2026-05-10T09:30:00Z"
}
```

Enumerations used across the platform:

Type	Values
Role	student, staff, admin
Severity	normal, warning, critical
Room status	empty, normal, busy, overloaded
Room type	classroom, lecture_hall, lab, library, office, shared space
Request category	maintenance, cleaning, IT, safety, medical

Appendix B: API Endpoints

Public routes need no token. Sensor routes need the device key header. Everything else needs a Cognito token.

Method	Endpoint	Access	Description
GET	/health	Public	Service status and version
POST	/events	Device key	Submit one sensor event
POST	/events/batch	Device key	Submit many events at once
GET	/me	Any signed in user	Current user and role
GET	/stats	Staff	Aggregated campus statistics
GET	/stats/energy	Staff	Energy consumption by building
GET	/rooms	Any signed in user	List rooms and their status
GET	/rooms/{roomId}	Any signed in user	One room in detail
GET	/events	Staff	Recent events, filterable

GET	/alerts	Staff	Open and historical alerts
PATCH	/alerts/{alertId}	Staff	Acknowledge or resolve an alert
GET	/service-requests	Any signed in user	List requests
POST	/service-requests	Student	Raise a request
GET	/service-requests/{id}	Any signed in user	One request in detail
PATCH	/service-requests/{id}	Staff	Update status or priority
GET	/thresholds	Admin	Current rule configuration
PUT	/thresholds	Admin	Change the rules at runtime

Appendix C: Deployment Guide

To rebuild CampusPulse in a fresh AWS account:

```
# 1. Create the state bucket (once per account)
sh infra/bootstrap-state.sh

# 2. Point the backend at your bucket
#   edit infra/providers.tf, set bucket = campuspulse-tfstate-<your account id>

# 3. Set your variables
cp infra/terraform.tfvars.example infra/terraform.tfvars

# 4. Build the Lambda binary
sh infra/build-lambda.sh

# 5. Create the infrastructure
cd infra && terraform init && terraform apply

# 6. Build and upload the dashboard
sh infra/deploy-dashboard.sh

# 7. Create the demo accounts
cd backend && go run ./cmd/setup
```

After that, pushing to main deploys automatically, provided the three GitHub secrets are set: AWS_ROLE_ARN, BUDGET_EMAIL and DEMO_PASSWORD.